

1) Pour commencer, j'ai fait les opérations générales :

Split d'un triangle en 3 :

But : insérer un sommet P dans la face $(a,b,c) \rightarrow (a,b,m), (b,c,m), (c,a,m)$.

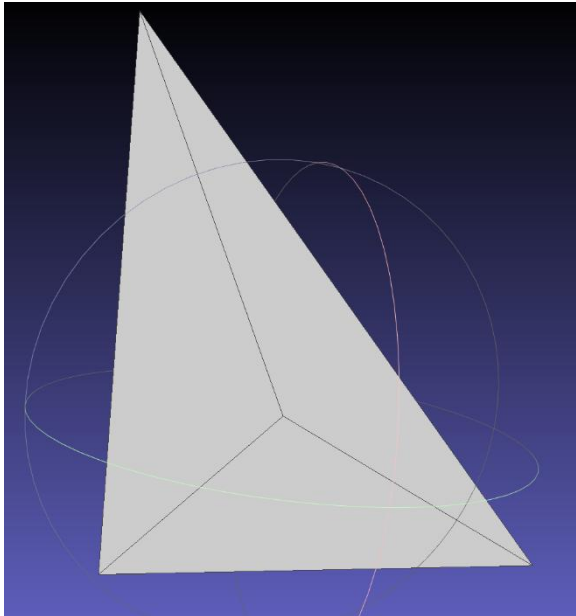


Figure 1 : Exemple d'un triangle auquel j'ai fait un `splitTriangle`.

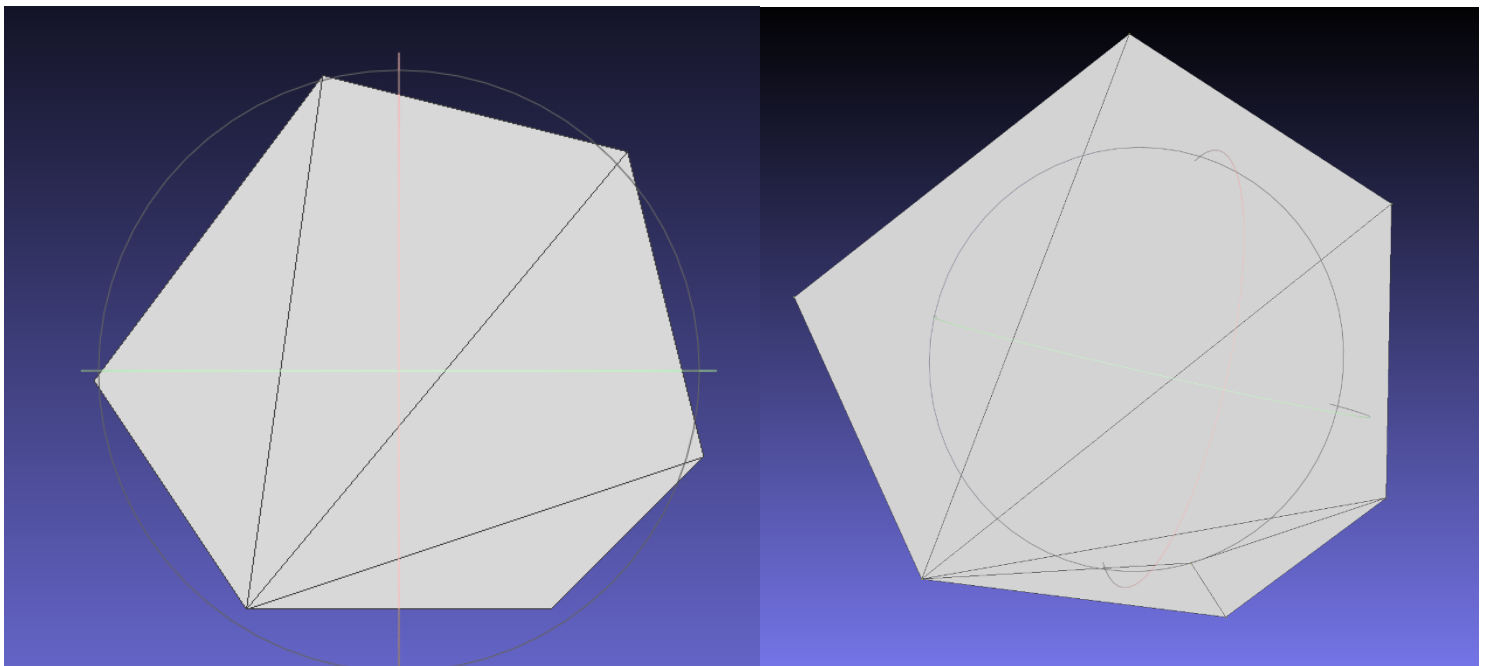


Figure 2 : Même chose que Figure 1 mais sur une forme un peu plus complexe, le `split triangle` est visible sur le bas de la forme de droite.

Split d'une arête en 2 :

But : insérer P au milieu de l'arête (i,j), divisant les 1 ou 2 triangles incidents.

- Sur $t=(a,b,k)$ où arête (i,j) est (a,b), je remplace par $(a,m,k) + (m,b,k)$.
- Si voisin t_n existe de l'autre côté : idem en miroir.

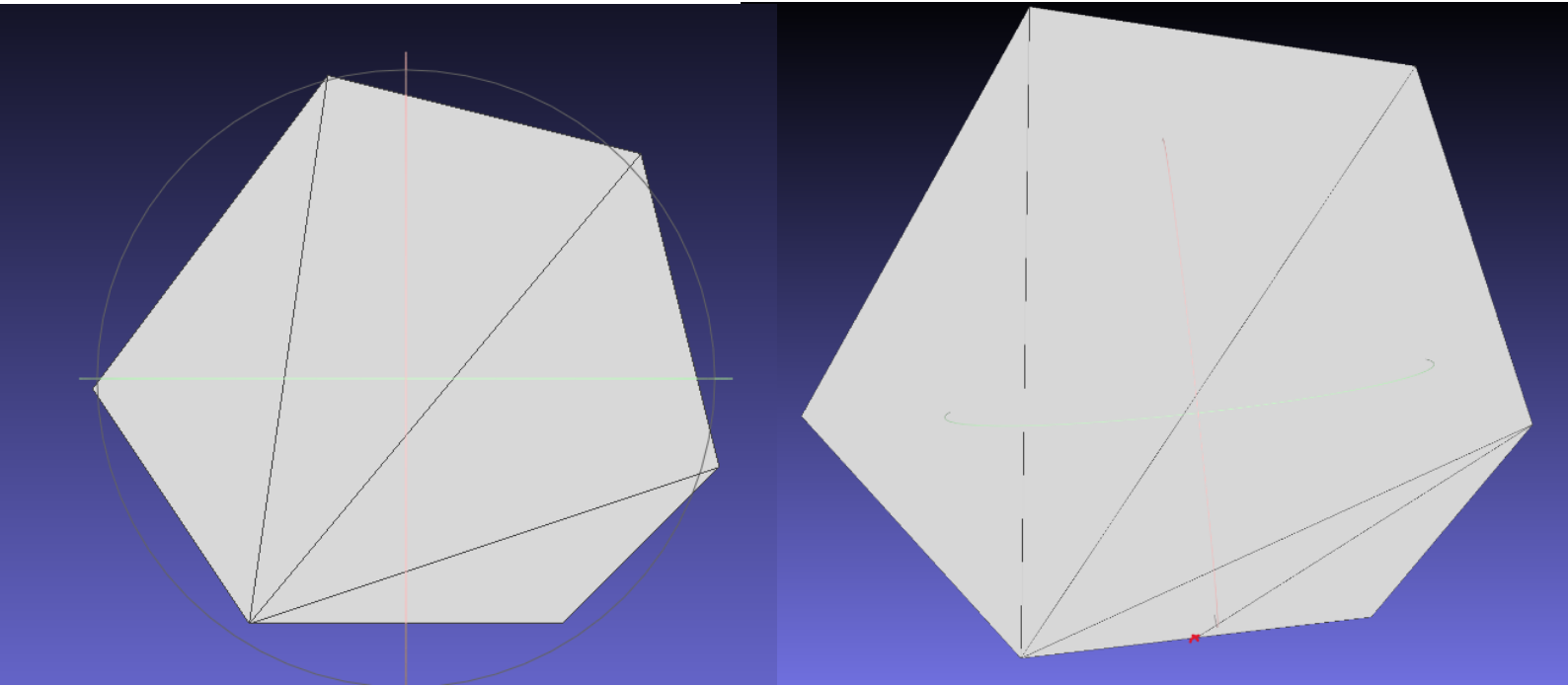


Figure 3 : Exemple d'un splitedge, forme de gauche l'objet original, forme de droite l'objet avec un split edge positionner en bas de la forme.

Flip d'une arête interne :

But : je veux remplacer l'arête diagonale (i,j) d'un quadrilatère convexe (i,k,j,l) par (k,l).

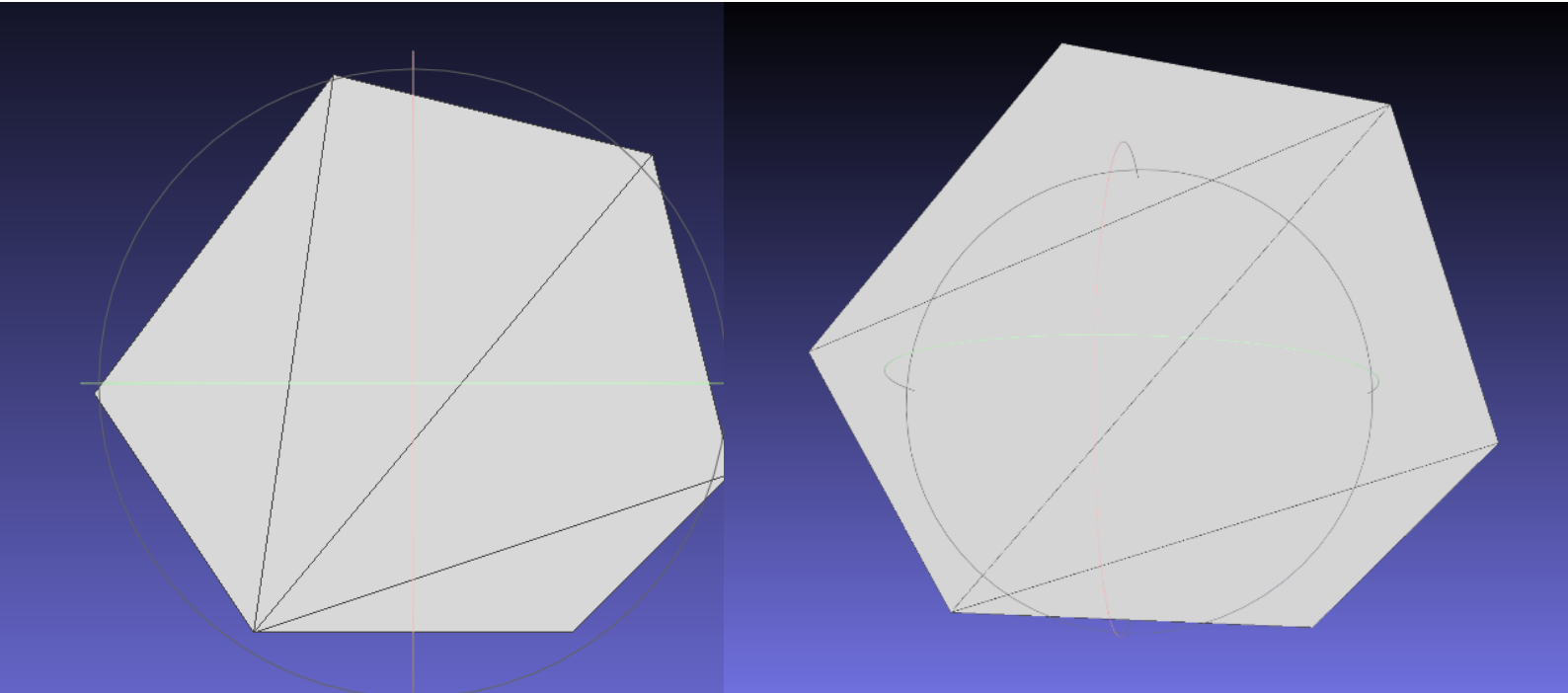


Figure 4 : Exemple d'un flip edge, la forme de droite représente l'objet après un flippedge de l'arête du haut.

2) Prédicats géométriques (2D)

Orientation (aire signée $\times 2$) :

$$\text{orient2D}(A,B,C) = (B_x - A_x) * (C_y - A_y) - (B_y - A_y) * (C_x - A_x)$$

Test "point dans triangle" :

Je regarde les trois orientations (A,B,P), (B,C,P), (C,A,P).

- Tous de même signe $\rightarrow$ **intérieur** (return +1)
- Un ou plusieurs nuls, et pas de signe contradictoire $\rightarrow$ **sur bord** (return 0)
- Sinon **extérieur** (return -1)

Insertion naïve d'un point en 2D :

But : Je veux insérer P (**sans** penser à la qualité).

Cas 1 – P est dedans/sur bord

- Parcours des triangles : inTriangle.
- Si dedans → splitTriangle.
- Si sur bord → splitEdge.

Cas 2 – P est dehors de l'enveloppe convexe

- Version 1 (vraiment naïf) : **je liste les arêtes de bord visibles** et je crée (i,j,P).
- Version 2 (avec point fictif) : j'utilise un **sommet fictif** relié à tout le bord, puis split/flip, puis je **retirer** le sommet fictif (sinon voir Figure 6).

(Avec la version 2, l'insertion est censée être Delaunay directement d'où l'avantage)

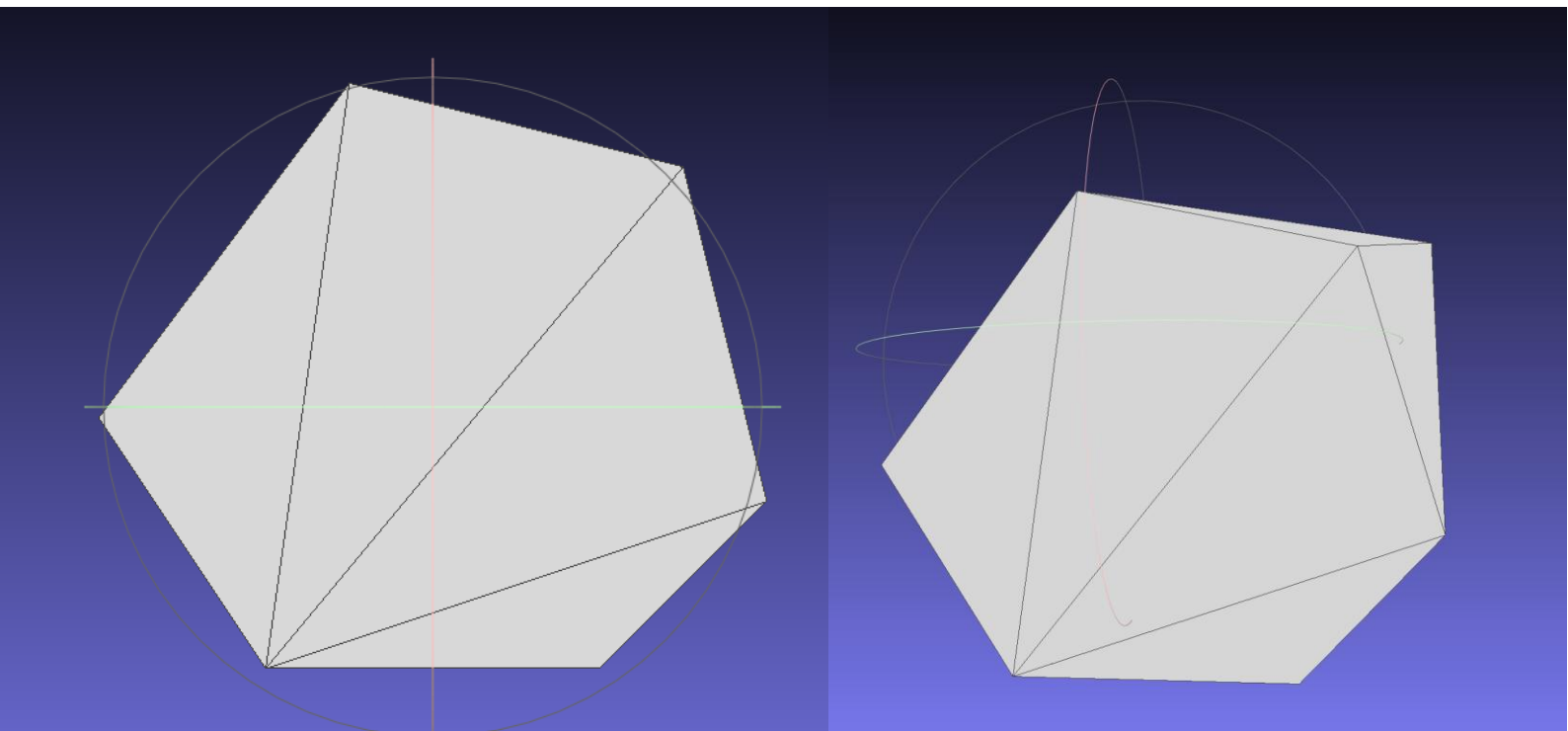


Figure 5 : Exemple d'une insertion d'un point n'était pas dans l'enveloppe convexe de l'objet.

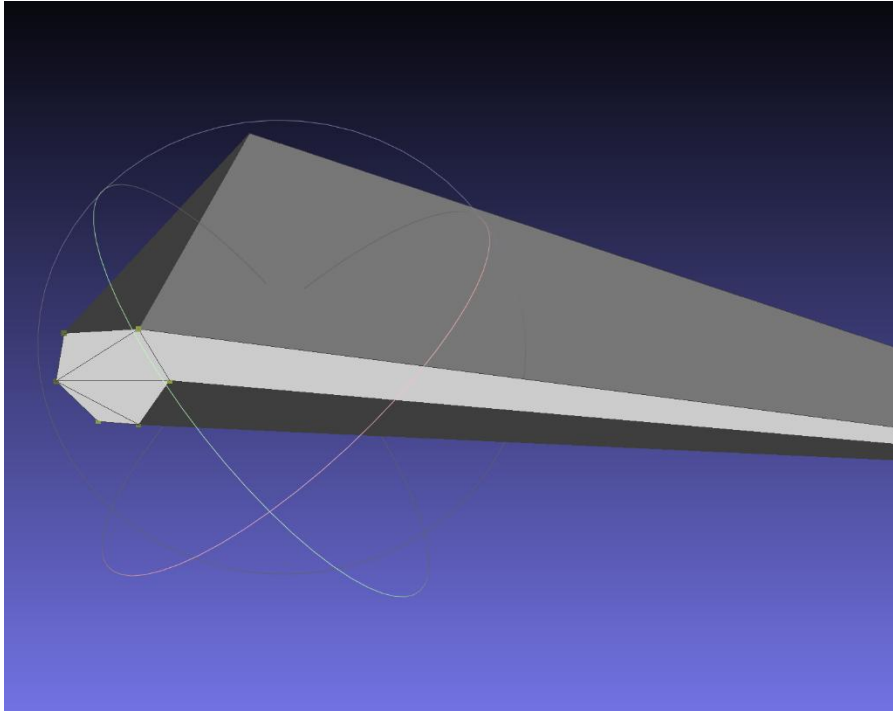


Figure 6 : Exemple d'erreur lors du tp, ici j'avais oublié de retirer le point fictif, aussi on peut voir des faces sombre car elles ne sont pas orientées dans le bon sens..

3) Delaunay

Test local Delaunay pour une arête (i,j) :

On a deux triangles adjacents (i,j,k) et (j,i,l) .

- Je teste que **L n'est pas à l'intérieur** du cercle circonscrit de (i,j,k) **et** que K n'est pas à l'intérieur de celui de (j,i,l) .

```
mantador@DESKTOP-1V9DVDH:~/M2/GAM/TP1$ ./test save.off
Arête non Delaunay trouvée entre sommets {2,0} dans le triangle 0
Arête non Delaunay trouvée entre sommets {0,2} dans le triangle 1
Total d'arêtes non Delaunay: 2
Total d'arêtes Delaunay: 10
```

Figure 7 : Exemple d'une sortie quand l'objet de base (ici l'objet qu'on utilise depuis le début save.off) n'est pas complètement Delaunay.

Lawson (global) — rendre toute la triangulation Delaunay :

- Je mets dans une pile **toutes les arêtes internes** (une seule fois).
- Tant que la pile n'est pas vide :
 - Pop (t,e) $\rightarrow$ arête (i,j) et son voisin tn.
 - Si **pas Delaunay** $\rightarrow$ flipEdge(t,i,j) $\rightarrow$ **re-pousser** les arêtes internes autour des deux nouveaux triangles.

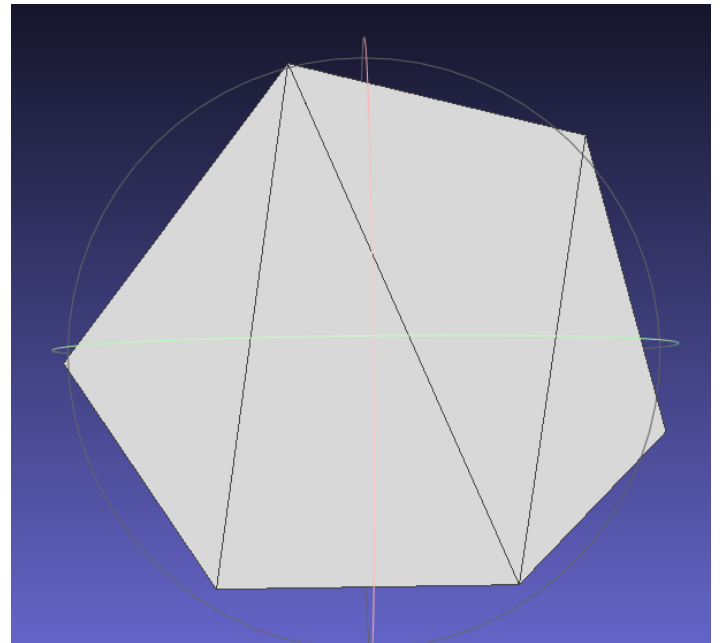
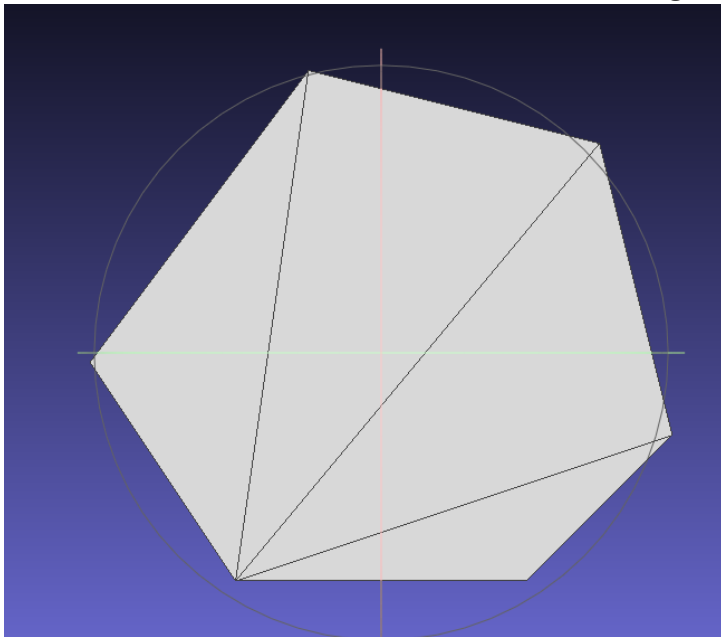


Figure 8 : Exemple d'une modification du maillage pour le rendre Delaunay.

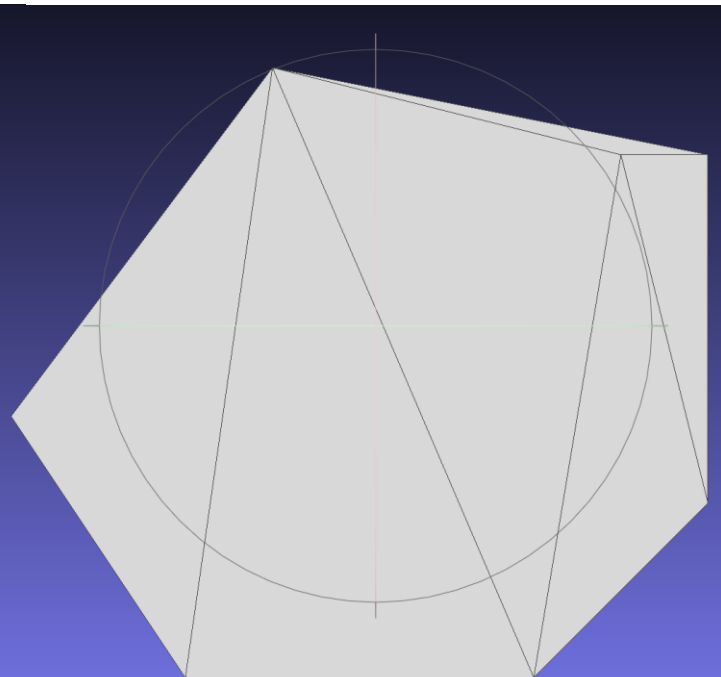
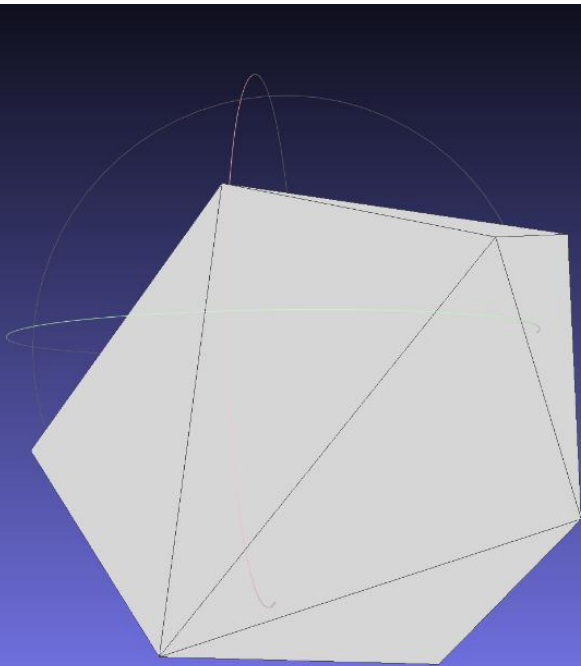


Figure 9 : Exemple d'une modification du maillage pour le rendre Delaunay (ici avec la version ou on a ajouté un point en dehors)

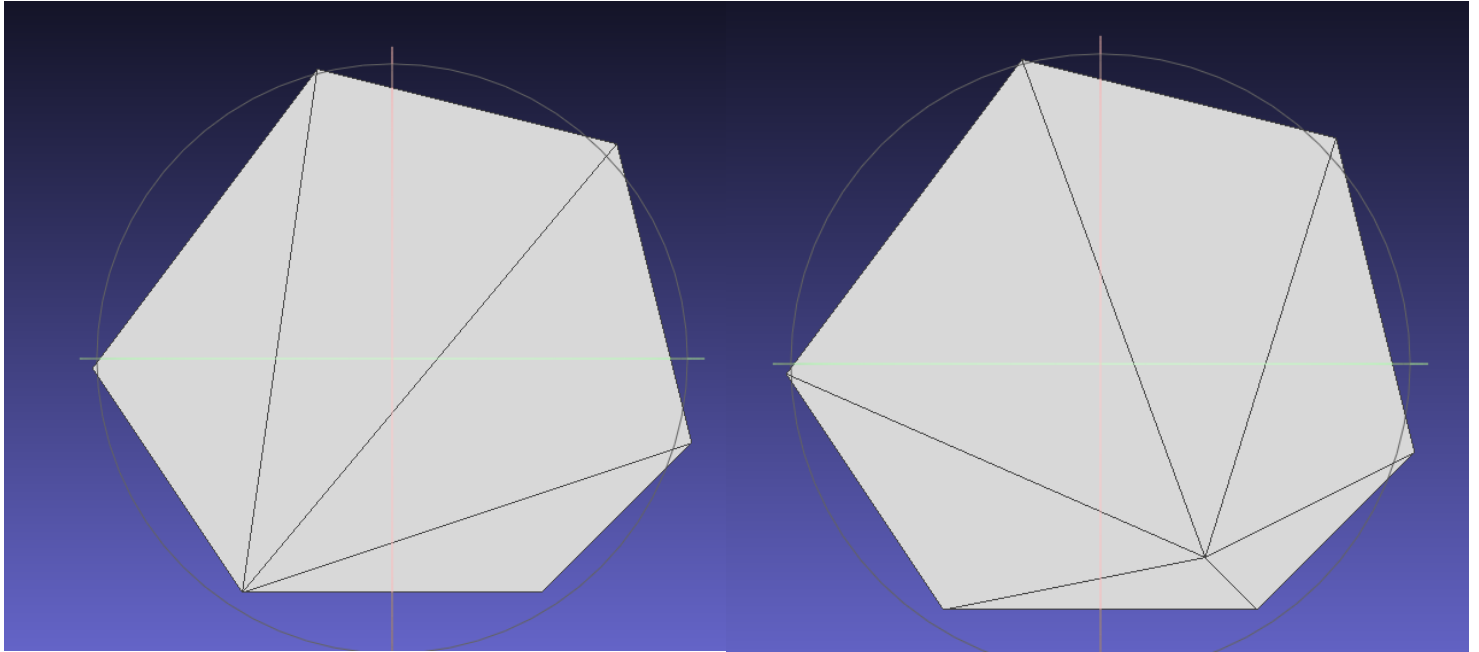


Figure 10 : Exemple d'une insertion dans l'objet tout en restant Delaunay.

```
mantador@DESKTOP-1V9DVDH:~/M2/GAM/TP1$ ./test after_insert_delaunay.off
La triangulation est entièrement Delaunay.
```

Figure 11 : Exemple d'une sortie lorsqu'on teste si le maillage est complètement Delaunay.

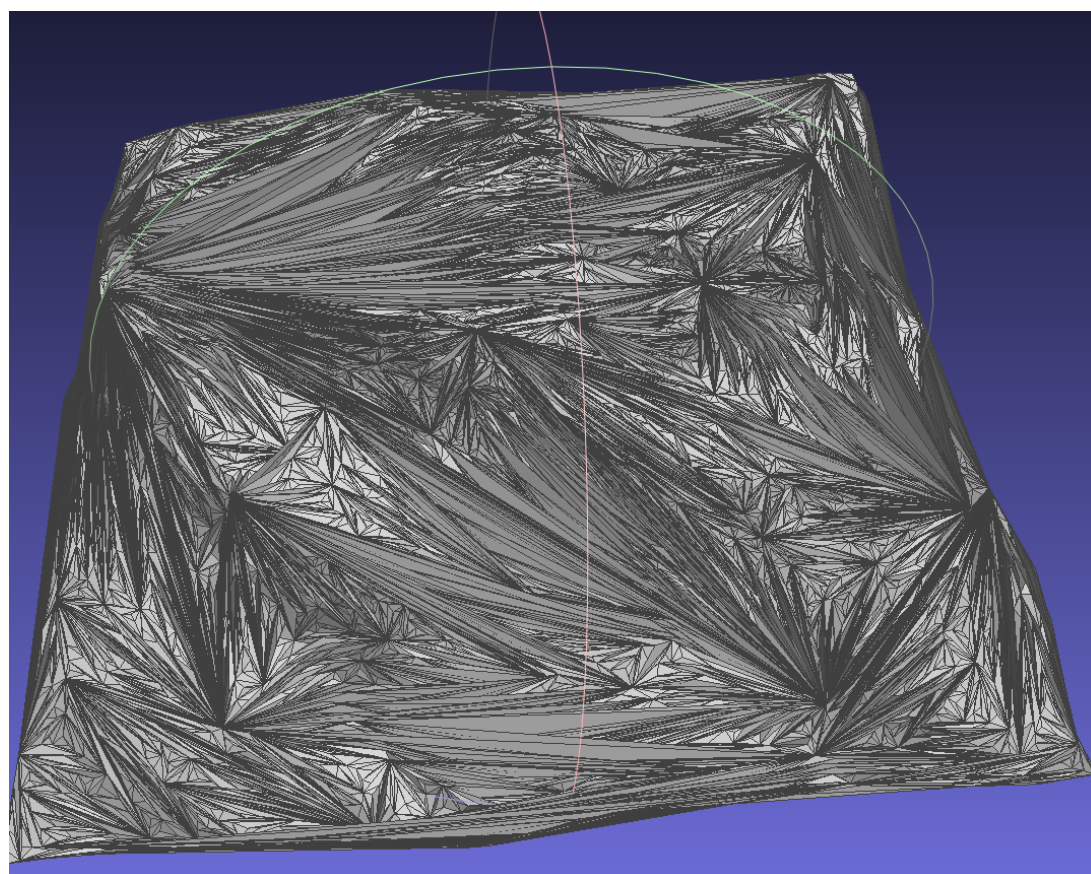
Terrain :

Je construis une triangulation 2D en conservant Z comme altitude, on va du coup créer le maillage à partir du nuage de point de alpes_poisson.

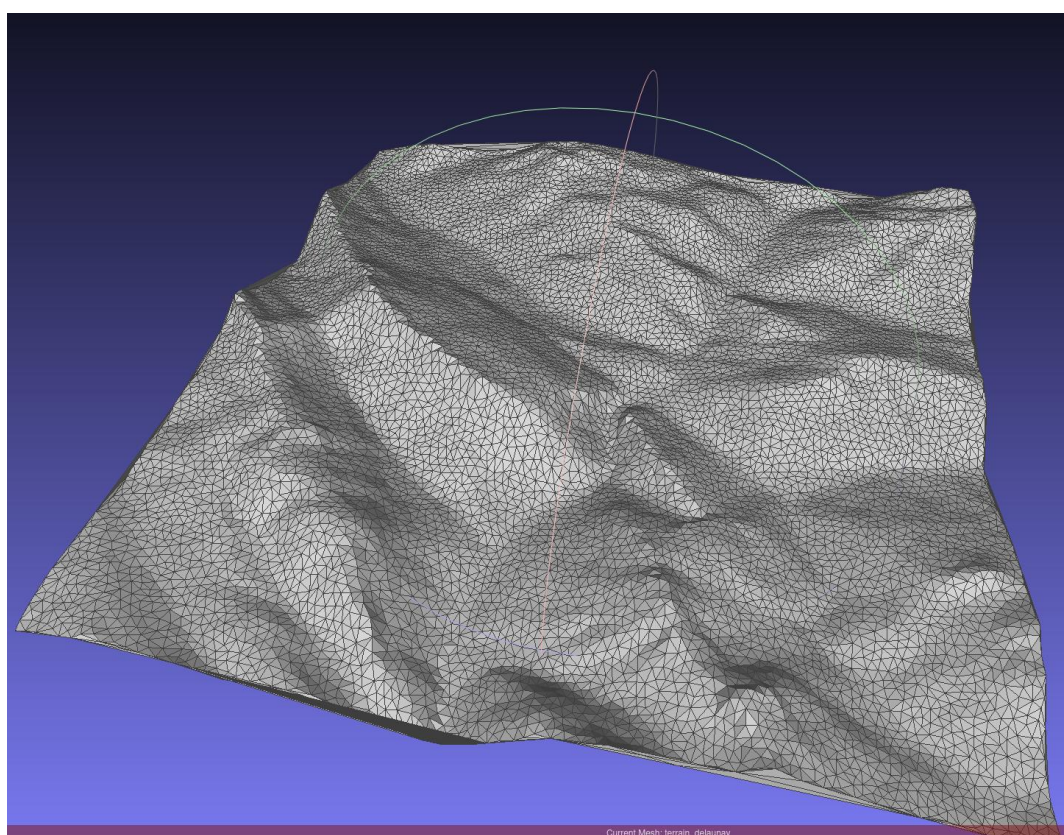
Dans le premier cas je construis la triangulation « comme elle vient » entre autres, je ne cherche pas à optimiser la qualité des triangles (c'était vraiment pour comparer les 2 versions).

Je pars d'un premier triangle et je fais l'insertion de point avec l'insert2Dnaive, donc ça va me créer une multitude de triangle vraiment moche (*cf Figure 12*), mais il faut noter que l'algo est nettement plus rapide que la version Delaunay (Lawson).

Pour la deuxième version je veux garantir la qualité en rendant la triangulation localement Delaunay, donc on parcourt les arêtes internes, si une arête n'est pas localement Delaunay (test inCircle) alors on flip (etc jusqu'à stabilisation). Ensuite on insère P comme dans la version naïve puis on légalise uniquement les arêtes opposées à P (flip locaux + propagation). L'algo est vraiment très long cependant...

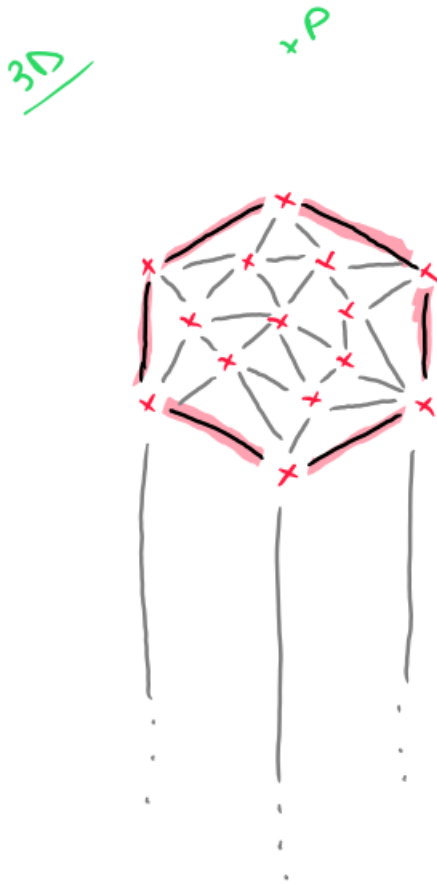


*Figure 12 :
Exemple du
maillage résultant
avec l'insertion 2D
naïve, on ne fait
pas le Delaunay
ici. (c'est très
moche)*

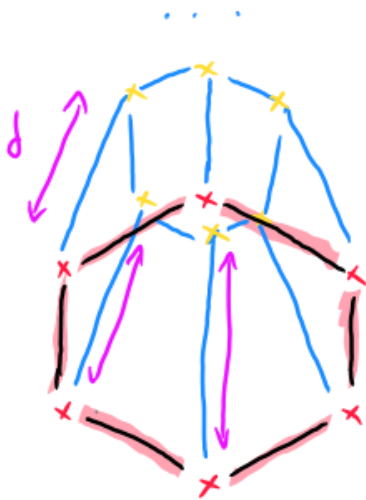


*Figure 13 : Exemple du maillage résultant avec l'algorithme de lawson pour l'insertion
des points. (c'est tout beau)*

Vu qu'on travaille sur des maillages 2D, sur un maillage 3D comme queen.off j'aurais bien vu ceci :



Seule les pts "x" sont visible de P.
Mais on va prendre que les pts visible
d'une enveloppe convexe des points visible
arêtes



Via cette fonction d'insertion, si on insert
en dehors cela nous donne une opportunité de
créer des formes comme des "boutons", "des simplos bossant
ou des cavités (si on creuse donc insert à l'intérieur de l'obj 3D
ou surélevement d'un terrain, d'une zone précise.
On peut faire beaucoup de choses...

Faut penser aussi à supprimer les triangles inutiles (contenu dans les nouveaux triangles) même si ça peut coûter..